GBST-BERT: does a learned tokenizer help a small DNA language model?



Practical work: pretraining on the human genome (hg38) and 18 prediction tasks

Presented by Omar Mohamed, K12103822

JOHANNES KEPLER UNIVERSITY LINZ

Problem Overview

- **Problem:** a DNA language model must first cut DNA into tokens – single bases or fixed k-mers. A fixed cut can split a motif in the wrong place.
- **Question:** does a learned tokenizer (GBST, from ChaRNABERT) help a small model learn the human genome and predict biology?
- **Relevance:** models that read DNA predict promoters, enhancers, splice sites and histone marks from sequence alone – a basis for interpreting genomes and variants.
- **State of the art:** Bio-xLSTM (2024) compares small DNA models – HyenaDNA, Mamba, xLSTM – on the 18-task Nucleotide Transformer benchmark.

Three ways to cut the same DNA

Single bases

T A T A A A G G C

fine detail, no grouping

Fixed 3-mers

TAT | AAA | GGC

can split a motif (TATA box)

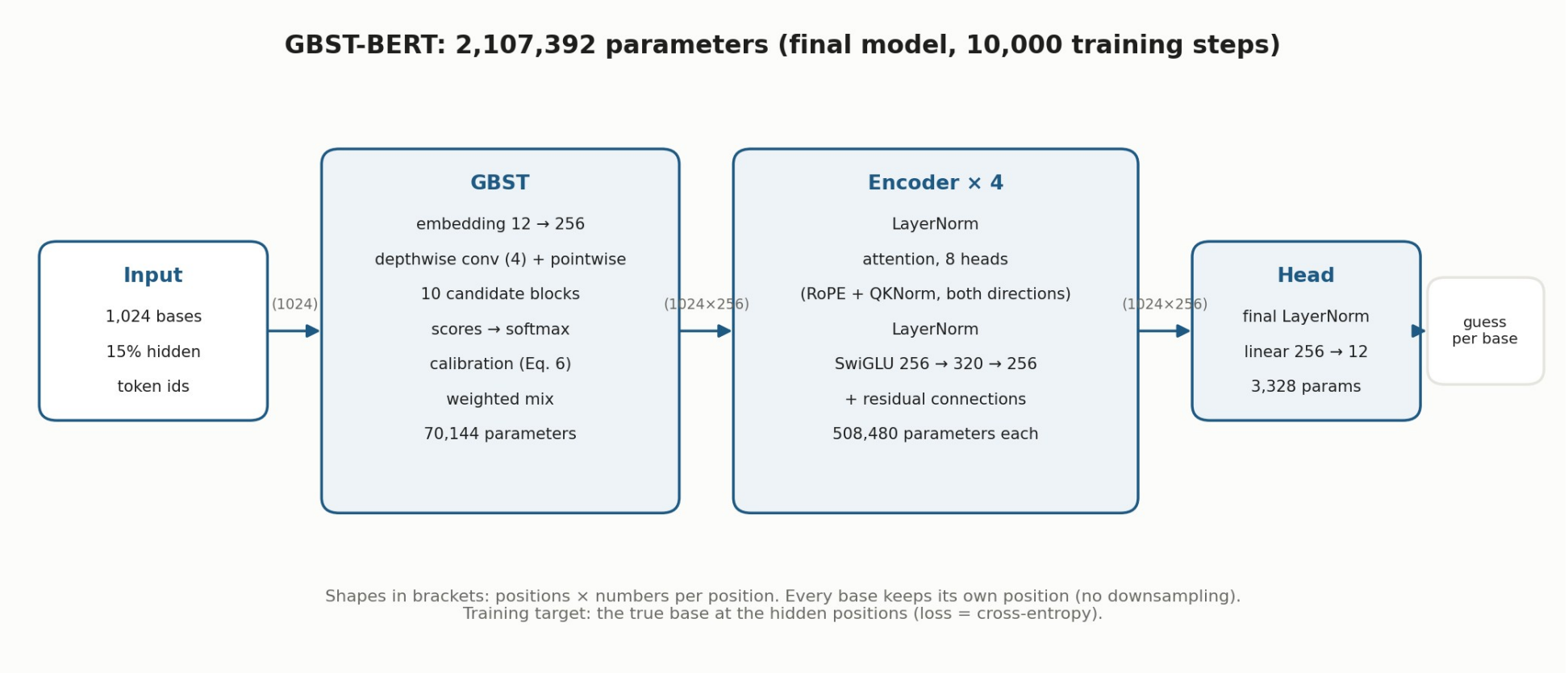
GBST (learned)

TATAAA | G | GC

block sizes 1–4 learned (illustration)

ChaRNABERT: Morales-Pastor et al. 2024, arXiv:2411.11808 · Bio-xLSTM: Schmidinger et al. 2024, arXiv:2411.04165

Method Interpretation



How it works: GBST mixes each base with its neighbours, scores candidate blocks of 1–4 bases and returns a weighted mix – one vector per base – which a bidirectional BERT encoder reads.

Why this approach

1

Learned, not fixed

GBST decides from the data how to group bases, while every base keeps its own position – a single-base change (SNP) still changes the input.

2

A clean comparison

Control model Transformer++: the 2 M Transformer of Bio-xLSTM, same size, same training, plain lookup table instead of GBST.

3

A fair benchmark

Pretraining and fine-tuning follow the Bio-xLSTM protocol exactly, so our numbers sit next to the published HyenaDNA, Mamba and xLSTM results.

2,107,392

parameters in GBST-BERT

70,144

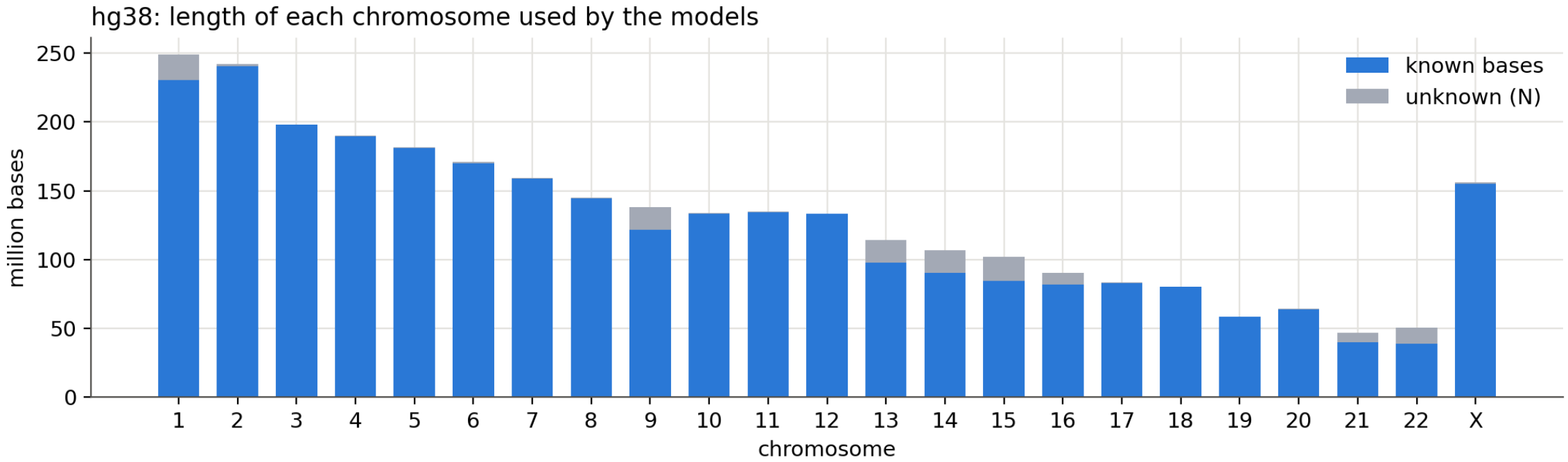
of them in GBST (3%)

2,036,992

parameters in Transformer++

JOHANNES KEPLER UNIVERSITY LINZ

Data: the human genome hg38



3.03 billion

bases, chromosomes 1–22 and X (4.0% unknown, N)

2.33 billion

distinct training bases (34,021 regions)

0.29 / 0.27

billion validation / test bases – never seen in training

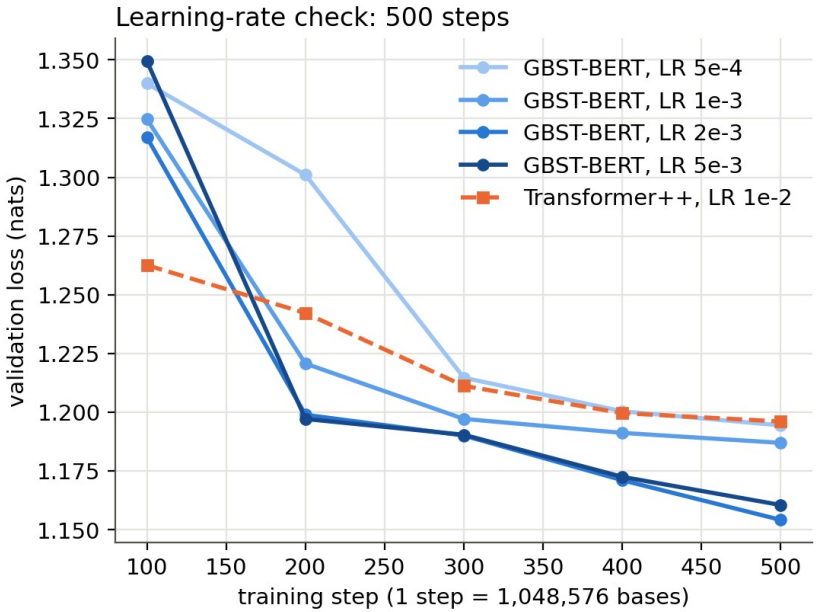
JOHANNES KEPLER UNIVERSITY LINZ

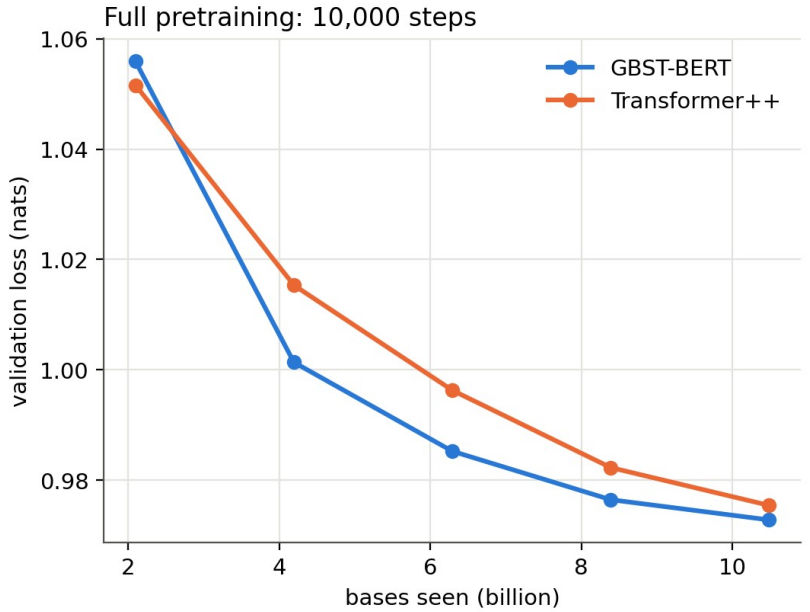
Pretraining: identical protocol for both models

Task	masked language modelling: hide 15% of bases, guess them
Batch	1,024 sequences × 1,024 bases per step
Length	10,000 steps = 10.5 billion bases per model
Optimiser	AdamW (0.9, 0.95), weight decay 0.1, cosine schedule
Learning rate	GBST-BERT 2e-3 (checked: 5e-4 … 5e-3); Transformer++ 1e-2
Hardware	4 GPUs (JKU student servers), 32-bit precision

Same data, same masks, same schedule: only the tokenizer differs. Settings from Bio-xLSTM App. C.5 and Tables A3–A4; the learning rate of GBST-BERT was chosen by a 500-step check, Transformer++ uses the paper's value.

Pretraining result: almost level at the end





56.44%

GBST-BERT: hidden bases guessed right (2,048 test sequences)

56.33%

Transformer++

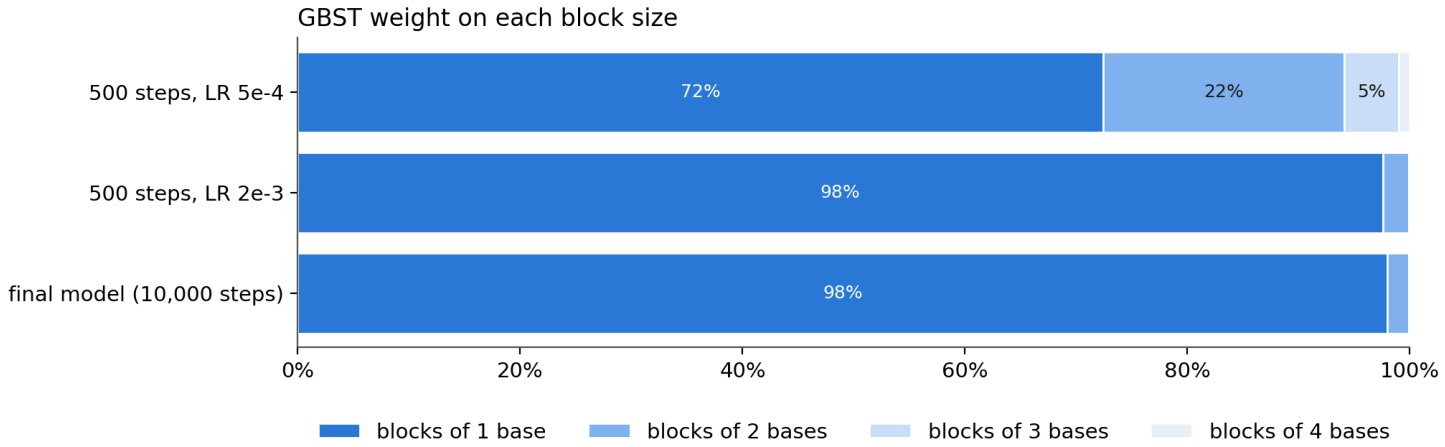
25%

random guessing

Left: 500-step learning-rate check – 2e-3 lowest loss. **Right:** full runs – GBST-BERT ahead from 4 to 8 billion bases, final validation loss 0.9728 vs 0.9755.

A 0.1-point gap from one pretraining run per model is too small to call. Test loss 0.9753 vs 0.9774.

Inside the model: GBST chose single bases



Interpretation: on human DNA, GBST does not learn longer “words”. It behaves like a single-base model with a small local window in front – a likely reason why the pretraining losses end so close.

98%

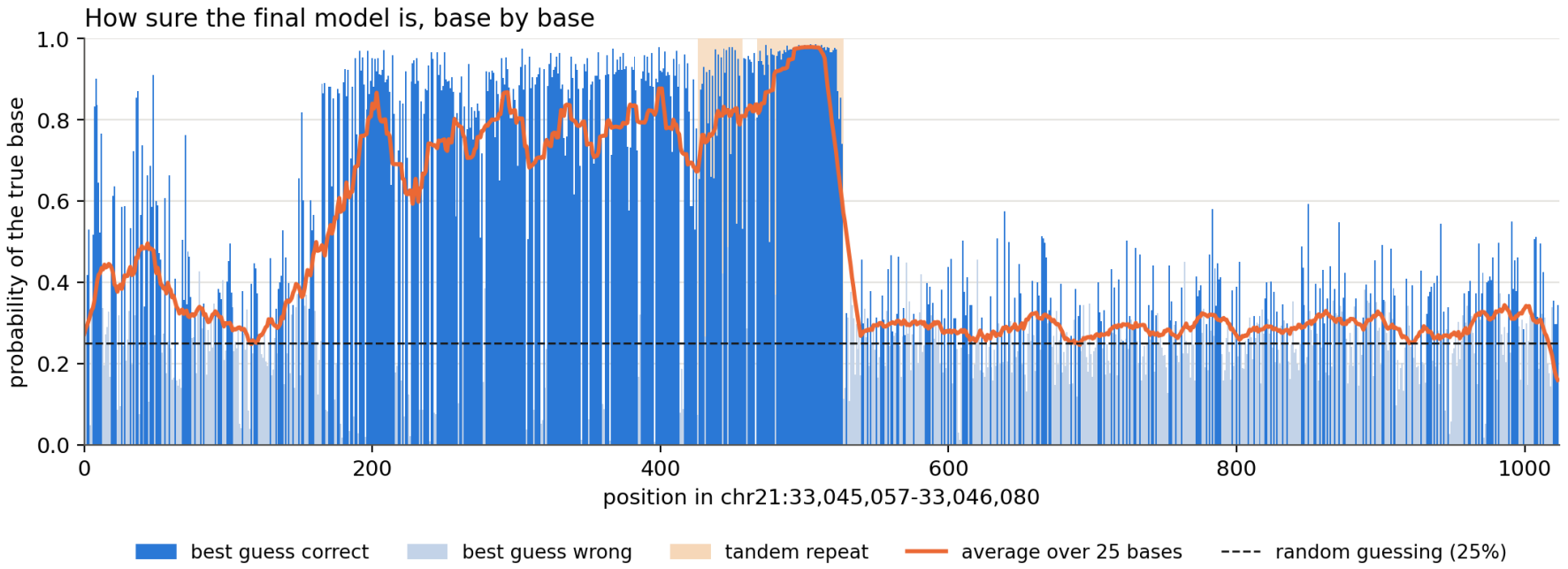
of GBST's weight is on single-base blocks in the final model

Early in training (LR 5e-4) it still used longer blocks: 72 / 22 / 5 / 1%.

Each base still sees its 3 neighbours through the convolution before the blocks.

Average over 8 hg38 sequences from different chromosomes, 15% of bases hidden.

Inside the model: it learned real DNA structure



chr21:33,045,057–33,046,080, every base hidden once. Shaded: tandem repeats such as (GAAA)n.

94.5%

of hidden bases right inside tandem repeats

50.8%

right in the rest of the window

25%

random guessing

Prediction tasks: the Nucleotide Transformer benchmark

10

Histone marks

Is this DNA on a nucleosome with a given mark (H3K4me3, …)? Score: MCC

2

Enhancers

Is it an enhancer, and which type? Score: MCC

3

Promoters

Is it a promoter (all, with / without TATA box)? Score: F1

3

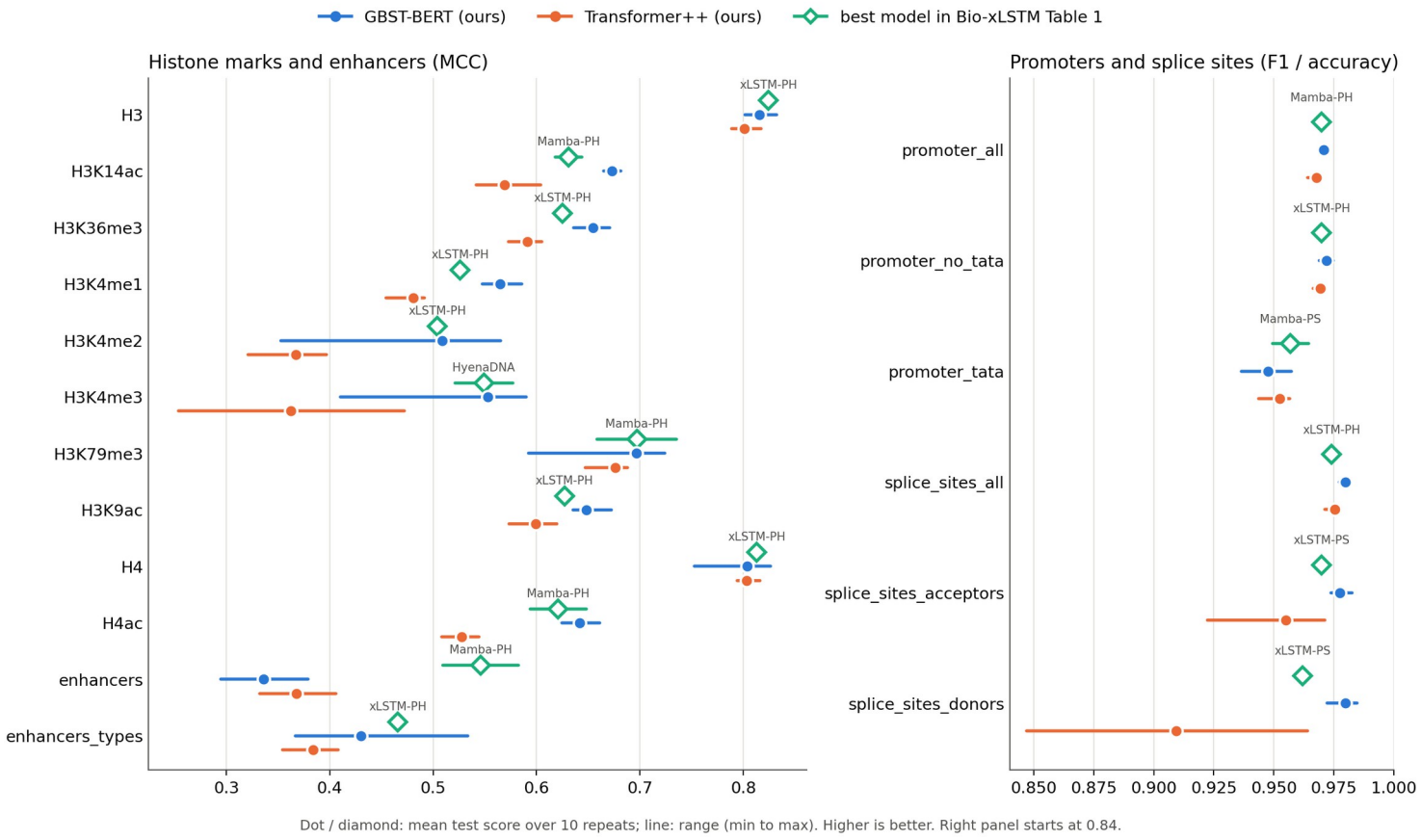
Splice sites

Donor, acceptor or neither? Score: F1 / accuracy

Same protocol as Bio-xLSTM: pretrained model + mean pooling + linear layer, whole model fine-tuned for 20 epochs, both DNA strands averaged, best validation epoch tested, **10 repeats per task** (360 runs). Per-task learning rate and batch from the paper (Table A5); on splice acceptors / donors these made GBST-BERT collapse, so 1e-3 and 5e-4 were also tried and chosen by validation score.

MCC: 0 = guessing, 1 = perfect. Data: InstaDeepAI/nucleotide_transformer_downstream_tasks.

Results: GBST-BERT is furthest right on most tasks



Dot / diamond: mean test score over 10 repeats; line: range (min to max). Higher is better. Right panel starts at 0.84.

How to read it

One row per task; further right is better.

Dot = mean of 10 repeats; line = worst to best repeat.

Blue GBST-BERT, **orange** Transformer++, green: best model of the paper.

Strongest: splice sites and histone marks.

Weakest: enhancers, for both of our models.

All models: histone marks (MCC)

Task	HyenaDNA	Mamba-PS	Mamba-PH	xLSTM-PS	xLSTM-PH	GBST-BERT	Transformer++
H3	0.779 ±0.037	0.799 ±0.029	0.815 ±0.048	0.796 ±0.014	**0.824** ±0.010	0.815 ±0.031	0.801 ±0.029
H3K14ac	0.612 ±0.065	0.541 ±0.212	0.631 ±0.026	0.570 ±0.008	0.598 ±0.017	**0.673** ±0.016	0.569 ±0.062
H3K36me3	0.613 ±0.041	0.609 ±0.109	0.601 ±0.129	0.588 ±0.019	0.625 ±0.010	**0.655** ±0.035	0.591 ±0.032
H3K4me1	0.512 ±0.024	0.488 ±0.102	0.523 ±0.039	0.490 ±0.012	0.526 ±0.001	**0.565** ±0.038	0.481 ±0.037
H3K4me2	0.455 ±0.095	0.388 ±0.101	0.487 ±0.170	0.489 ±0.024	0.504 ±0.012	**0.509** ±0.212	0.367 ±0.076
H3K4me3	0.549 ±0.056	0.440 ±0.202	0.544 ±0.045	0.520 ±0.019	0.537 ±0.012	**0.553** ±0.180	0.362 ±0.219
H3K79me3	0.672 ±0.048	0.676 ±0.026	**0.697** ±0.077	0.662 ±0.011	**0.697** ±0.007	**0.697** ±0.132	0.676 ±0.041
H3K9ac	0.581 ±0.061	0.604 ±0.048	0.622 ±0.030	0.622 ±0.013	0.627 ±0.008	**0.648** ±0.037	0.599 ±0.046
H4	0.763 ±0.044	0.789 ±0.020	0.811 ±0.022	0.793 ±0.011	**0.813** ±0.008	0.804 ±0.074	0.803 ±0.022
H4ac	0.564 ±0.038	0.525 ±0.240	0.621 ±0.054	0.558 ±0.018	0.583 ±0.014	**0.642** ±0.037	0.528 ±0.036

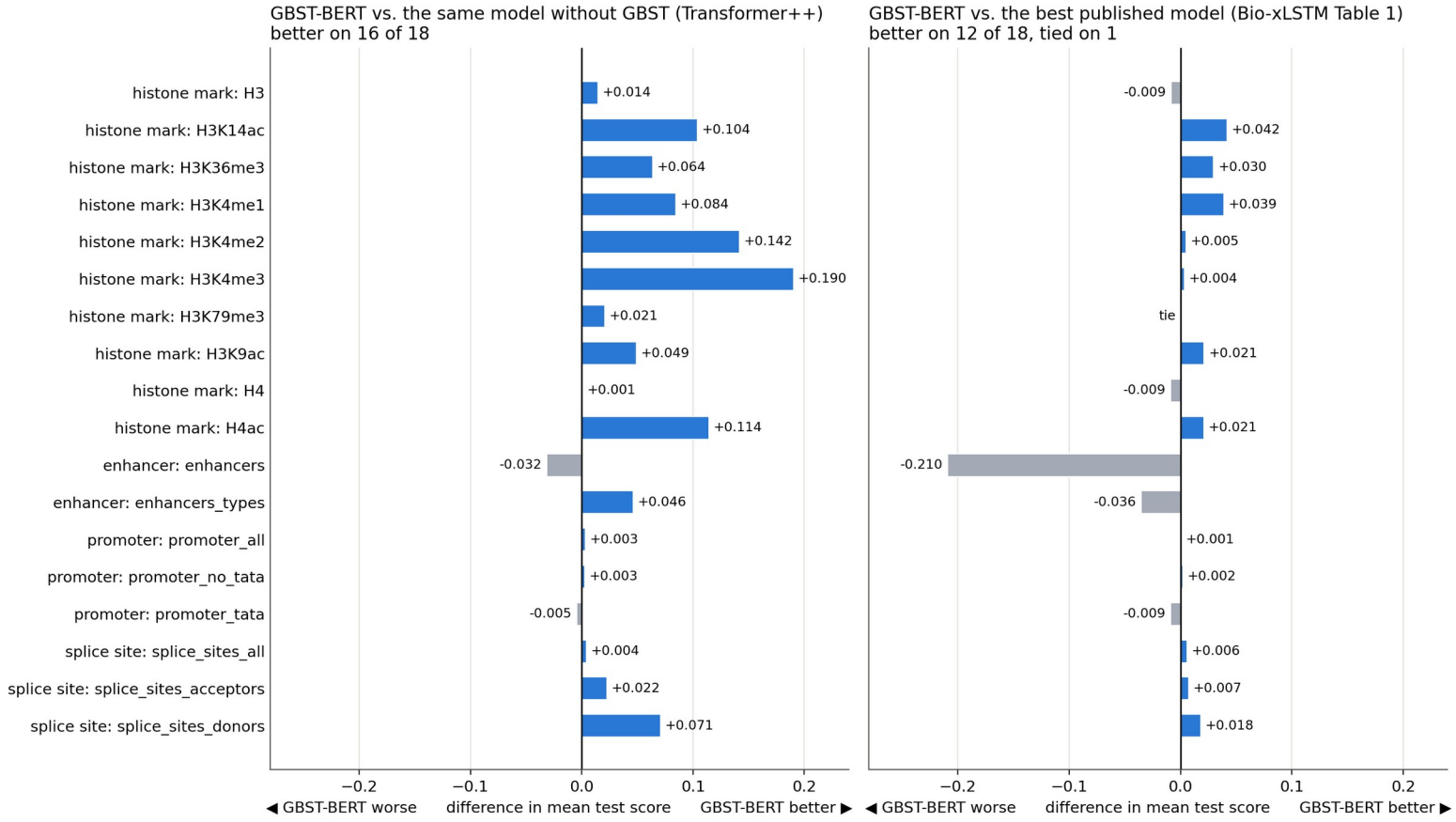
Mean over 10 repeats ± range (best − worst). Highlighted: best mean (ties marked for each). Columns 1–5: Bio-xLSTM Table 1.

All models: enhancers, promoters and splice sites

Task	HyenaDNA	Mamba-PS	Mamba-PH	xLSTM-PS	xLSTM-PH	GBST-BERT	Transformer++
enhancers	0.517 ±0.117	0.491 ±0.066	**0.546** ±0.073	0.375 ±0.030	0.545 ±0.024	0.336 ±0.085	0.368 ±0.074
enhancers_types	0.386 ±0.185	0.416 ±0.095	0.439 ±0.054	0.444 ±0.046	**0.466** ±0.011	0.430 ±0.167	0.384 ±0.053
promoter_all	0.960 ±0.005	0.967 ±0.004	0.970 ±0.004	0.962 ±0.002	0.967 ±0.001	**0.971** ±0.003	0.968 ±0.006
promoter_no_tata	0.959 ±0.011	0.968 ±0.006	0.968 ±0.010	0.963 ±0.002	0.970 ±0.001	**0.972** ±0.006	0.969 ±0.004
promoter_tata	0.944 ±0.040	**0.957** ±0.015	0.953 ±0.016	0.948 ±0.006	0.952 ±0.005	0.948 ±0.021	0.953 ±0.013
splice_sites_all	0.956 ±0.011	0.927 ±0.021	0.940 ±0.027	0.965 ±0.006	0.974 ±0.004	**0.980** ±0.004	0.975 ±0.006
splice_sites_acceptors	0.958 ±0.010	0.936 ±0.077	0.937 ±0.033	0.970 ±0.005	0.953 ±0.008	**0.977** ±0.009	0.955 ±0.049
splice_sites_donors	0.949 ±0.024	0.874 ±0.289	0.948 ±0.025	0.962 ±0.004	0.951 ±0.005	**0.980** ±0.012	0.909 ±0.117
Best on (of 18)	**0**	**1**	**2**	**0**	**4**	**13**	**0**

MCC for enhancers, F1 for promoters and splice acceptors / donors, accuracy for splice_sites_all. Last row: all 18 tasks, ties counted for each model.

The GBST effect, task by task



16 / 18

tasks where GBST-BERT beats the same model without GBST

12 + 1 tie

tasks where it beats or ties the best published model

Biggest gains: H3K4me3 +0.19, H3K4me2 +0.14.
Biggest loss: enhancers −0.21 vs the paper.

Results Interpretation

- **Pretraining:** both models learned human DNA (56% vs 25% chance); the 0.1-point gap is too small to call.
- **Prediction:** the difference is clear – GBST-BERT beats Transformer++ on 16 of 18 tasks and has the best mean of all seven models on 13 of 18.
- **Why?** GBST chose single bases (98%), so the gain likely comes from its local window and mixing layer – not yet proven, the models also differ in normalisation.
- **Limits:** enhancers are weak for both models; some repeats trained poorly (wide ranges); fine-tuning settings were the paper's, not tuned for our models.

16 / 18

better than the same model without GBST

13 / 18

best mean of 7 models (xLSTM-PH: 4)

0.980

splice donor F1 (best paper model 0.962)

GBST-BERT: advantages and disadvantages

Advantages

- Better than the same model without GBST on 16 of 18 tasks
- Best mean of 7 models on 13 of 18, all 3 splice tasks
- Keeps single-base resolution: a SNP still changes the input
- Small cost: 70 k parameters, 3% of the model
- No tokenizer to choose by hand; weights can be inspected

Disadvantages

- About 18% slower to fine-tune (962 s vs 816 s on H3)
- Less stable: collapsed at high learning rates on splice tasks
- On DNA it chose single bases – no longer “words” learned
- Weak on enhancers, like our other model
- Almost no gain in pretraining loss (0.1 points)

What was challenging

Genome reader bug

Parallel data readers shared one file and read wrong bases (up to 33%). Found after a crash, measured, fixed with a test.

Benchmark loader bug

Every task silently loaded the data of all 18 tasks. Fixed: one file per task.

Unstable fine-tuning

The paper's learning rates collapsed GBST-BERT on splice tasks. Re-chosen by validation score only.

Long runs on shared GPUs

10.5 billion bases per model, 384 fine-tuning runs; a full disk and a server error stopped the queue – fixed with checks and retries.

The first two were bugs in the original repository; every fix is documented on GitHub.

How to get better predictions

	What to do – and how	Why (from our results)
1	**Tune fine-tuning for our model** Search learning rate and batch per task, choose by validation score: `LRS="1e-4 3e-4 1e-3" BATCH_SIZES="64 128" bash run_nucleotide_charna.sh`	Splice donors: 0.663 → 0.980 by the learning rate alone
2	**Average several fine-tuned models** Ensemble: average the predictions of the 10 fine-tuned repeats of a task `all 384 fine-tuned models are saved (GitHub release)`	Some single repeats failed (ranges up to ±0.21)
3	**Pretrain longer** More steps (and, if memory allows, a larger model): `MAX_STEPS=20000 bash run_pretrain_charna.sh`	Validation loss was still falling at 10,000 steps
4	**Give the model longer DNA** Pretrain and fine-tune on longer pieces, for long-range signals: `SEQLEN=2048 bash run_pretrain_charna.sh`	Enhancers are the biggest gap: −0.21 vs the paper

Only the effect of the learning rate (1) was measured; the gains of 2–4 are expected, not yet tested. Any new run: same repeats, choose by validation score, report every result.

Conclusion and next steps

With 3% more parameters, a learned tokenizer made a small DNA model better at 16 of 18 biology tasks

– strongest on splice sites and histone marks, weak on enhancers.

1

Clean GBST test

Same model, GBST replaced by a lookup table – proves the gain comes from GBST

2

Tune per model

Learning-rate search for fine-tuning, as the paper did; fixes the wide ranges

3

Longer DNA context

Longer pretraining or longer pieces, to address enhancers

Code, models, notebook and results: github.com/OMAR622-ai/dna-xlstm-charna (branch charna-gbst)